Weighted Panel Forecasting on the Kaggle Hedge Fund Dataset

Introduction and exploratory data analysis

What this PDF covers

This section introduces the forecasting task, explains the panel structure and evaluation logic, and uses EDA to justify what kinds of models are defensible later.

Scope. Only the introduction and EDA story is included here. Classical and deep model results are intentionally left for the next section of the project.

Team
[Student Name 1], [Student Name 2], [Student Name 3]

Roll Numbers
[Roll No. 1], [Roll No. 2], [Roll No. 3]

Course
Applied Forecasting Methods

Faculty
[Faculty Name]

Institute
DA-IICT

Date
April 2026

Roadmap

OVERVIEW

Takeaway. The flow is simple: define the problem, define one series correctly, explain the split and metric, then use EDA to narrow the later modeling space.

1. Problem → 2. Series Key → 3. Data Scope → 4. Split + Metric → 5. EDA Findings → 6. Implications

Focus of this PDF

- Why the task is a weighted panel problem instead of one ordinary time series.
- Why chronology, horizon, and series identity all matter at the same time.
- Which EDA findings directly change later modeling decisions.

Not covered yet

- No classical forecast comparison in this PDF.
- No deep-learning result comparison in this PDF.
- This closes exactly at the handoff from EDA into modeling.

Problem Statement

OVERVIEW

Takeaway. This is a weighted panel forecasting problem: many related series evolve over time, and their forecast errors do not contribute equally to the final score.

Task definition

- predict `y_target` at future `ts_index` values
- do this for many anonymized series, not just one
- each row carries a competition `weight`
- forecasts are evaluated on later timestamps, not shuffled rows

Why this is harder than one series

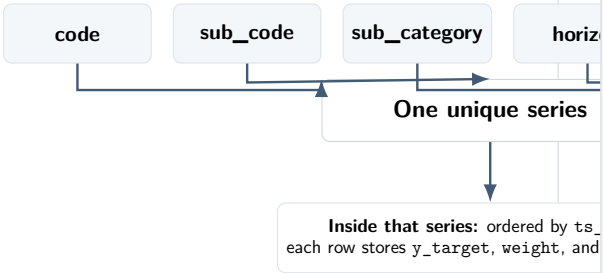
- different series have different lengths, volatility, and scale
- the target is heavy-tailed, so rare moves matter
- weights are extremely concentrated, so some errors dominate
- horizons `1, 3, 10, 25` define different forecasting regimes

How One Series Is Defined

OVERVIEW

Takeaway. One exact series is identified by `code + sub_code + sub_category + horizon`. `ts_index` then orders the observations inside that series.

Series-key schema



What each field means

Example key
`SJZP0OVU / OYJGNSQK / NQ58FVQM / H25`

Field	Meaning
`code`	primary identifier
`sub_code`	secondary identifier
`sub_category`	categorical grouping
`horizon`	forecast horizon
`ts_index`	time order within a series
`y_target`	continuous target value
`weight`	row importance in scoring

Dataset Scope And Length Rules

OVERVIEW

Takeaway. The panel is large, heterogeneous, and uneven in series history, so later modeling has to be constrained by explicit length rules rather than applied blindly to every series.

TRAIN ROWS	TEST ROWS	FEATURES	TOTAL SERIES
5.34M	1.45M	86	36,923

Panel coverage

Measure	Value
Codes	23
Train sub-codes	180
Test sub-codes	47
Sub-categories	5
Horizons	1, 3, 10, 25
Train timeline	`1..3601`

Length and split rules used later

- **Length** = number of train timestamps available for one exact series.
- **Minimum series length** = 120 points.
- **Minimum train length** = 96 points.
- **Minimum validation length** = 24 points.
- **Lookback** = 24 recent target values.

Timeline, Split, And Metric

OVERVIEW

Takeaway. Chronology is preserved twice: once at the global train-versus-test boundary, and again inside each eligible series through an 80/20 chronological train-validation split.

Timeline view of the forecasting problem

Global train partition `1..3601`	Held-out test `3602..4376`

Per-series chronological train 80% of an eligible series	Validation 20%

Why the split matters

- Random folds would leak future information into earlier states.
- The held-out test is a later time suffix, so training must respect order.
- The 80/20 per-series split gives later

Weighted skill score

$$\text{score} = \sqrt{1 - \text{clip}_{0,1}\left(\frac{\sum_i w_i (y_i - \hat{y}_i)^2}{\sum_i w_i y_i^2}\right)}$$

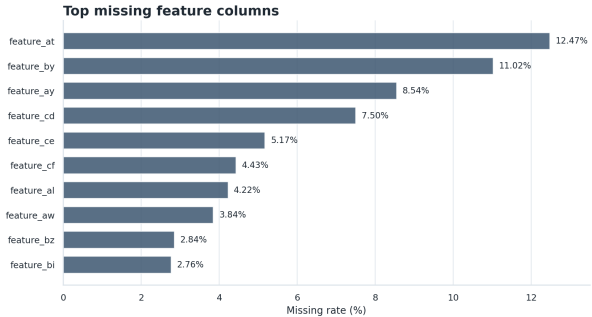
A higher score is better. The weights w_i mean

Missing Values

EDA

Takeaway. Missingness exists in 48 columns, but the maximum missing rate is only about 12.47%, so the main issue is selective feature reliability rather than catastrophic data loss.

Top missing features by rate



Top missing columns

Column	Rate
feature_at	12.47%
feature_by	11.02%
feature_ay	8.54%
feature_cd	7.50%
feature_ce	5.17%

Modeling implication

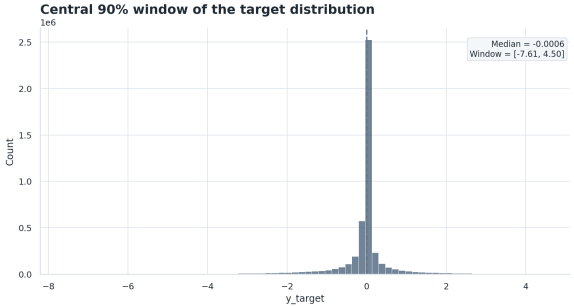
Feature reliability should be checked before using the full feature set, but the missing-

Target Distribution

EDA

Takeaway. The target is centered near zero, but the full distribution is strongly heavy-tailed, so the slide zooms into the central 90% window for readability while keeping tail statistics explicit.

Central mass of `y_target`



Summary statistics

Measure	Value
Median	-0.0006
1% quantile	-82.80
5% quantile	-7.61
95% quantile	4.50
99% quantile	62.92
Full min/max	$[-2201.88,\ 2314.41]$

Most observations sit close to zero, but tail events remain large enough to distort naive error summaries.

Representative Anchor Series

EDA

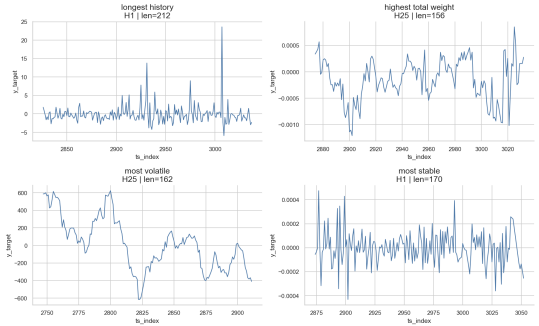
Takeaway. The project uses four anchor series to teach the range of behavior in the panel: one high-weight case, one long-history case, one stable case, and one volatile case.

Why these four series

- **High weight** | H25 | len 156: metric-sensitive, near-flat case
- **Long history** | H1 | len 212: richest lag context
- **Stable** | H1 | len 170: precision-dominated case
- **Volatile** | H25 | len 162: strongest instability

These are teaching anchors, not a claim that four series summarize the entire panel.

Representative series plots

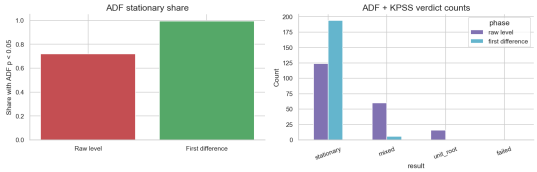


Stationarity And Differencing

EDA

Takeaway. On the sampled eligible series, the ADF stationary share rises from 69% in raw levels to 100% after first differencing, so differencing is not cosmetic here; it changes the data regime.

Raw levels versus first differences



How to read ADF and KPSS

- **ADF** asks whether a unit-root style non-stationary pattern is present.
- **KPSS** asks whether a stationary level assumption is reasonable.
- Using both tests prevents us from over-claiming stationarity from one test alone.

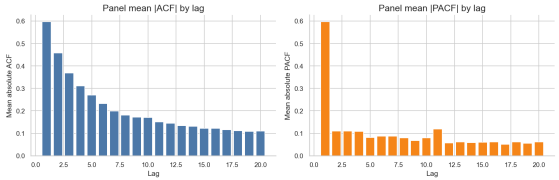
Later classical models should be al-

Panel Lag Structure

EDA

Takeaway. The panel-level lag summary shows that early lags dominate, especially lags 1 through 5, which supports compact autoregressive memory rather than long, diffuse dependence.

Mean absolute ACF and PACF by lag



What this means

- ACF captures persistence carried from recent history.
- PACF isolates each lag's direct contribution after shorter lags are accounted for.
- Both curves decay quickly, so short-memory structure is the main signal.

Low-order AR, ARMA, differenced ARIMA, and short target-history windows are all supported by this

Libraries Used And Why

EDA

Takeaway. The notebook stack is deliberately practical: use pandas and pyarrow for large panel data, matplotlib and seaborn for readable EDA, and statsmodels for time-series diagnostics.

Data and IO

- `pandas`: grouped summaries, tables, manifests
- `numpy`: arrays, quantiles, lag calculations
- `pyarrow`: large parquet loading
- `pathlib`, `json`: reproducible artifacts

Visualization

- `matplotlib`: final EDA figures
- `seaborn`: high-level statistical plotting

The deck reuses only the strongest visuals instead of dumping raw notebook outputs onto slides.

Diagnostics

- `statsmodels`: ADF, KPSS, ACF, PACF
- Enough for the introduction and EDA section

Later modeling notebooks additionally use forecasting routines from `statsmodels` and recurrent models from `torch`, but this PDF stops before those result sections.

EDA Synthesis

EDA

Takeaway. The EDA does not just describe the dataset. It narrows the later modeling space by telling us what assumptions are plausible and what comparisons would be misleading.

What we learned about the data

- The task is a weighted panel problem, not one ordinary time series.
- Missingness is selective rather than catastrophic.
- The target is centered near zero, but heavy tails remain important.
- Representative series expose large differences in stability, history length, and weight.

What this changes for modeling

- Chronology must be preserved in every split.
- Weights must remain central when reading scores.
- Differencing should be available where levels are not stable enough.
- Short-lag, compact-memory models are more defensible than arbitrarily long histories.

Transition

With the task, series definition, split logic, and EDA now established, the next project section can move into classical and deep forecasting with a much clearer methodological baseline.